

RS+BCH 级联码课题

需求澄清后的技术方案与执行计划

陈诗阳

基于 IEEE TIT 2025 论文 «Efficient Ordered Statistics Decoding of BCH Codes Without Gaussian Elimination»

<https://github.com/a-yeyang/efficient-osd-bch-no-ge>

2026-07-23

文档目的

根据老师对之前 6 个对齐问题 (Q1–Q6) 的答复 (R1–R6)，本文档整理出：

1. 逐条回答的技术解读和对整体方案的影响

2. 因回答衍生出的三个新技术追问，请老师再指点

3. 更新后的工作量估算和分阶段执行计划

1 老师答复的逐条解读

1.1 R1 ——内码单独 / 内外码都用 Lagrange 两种方案并列

R1 原文：请尝试内码单独、内外码两者都用 Lagrange 插值的方案。

解读：R1 确认了“Lagrange 共享”是课题的核心创新点，并明确要求做消融对比 (ablation) 来量化共享带来的收益：

方案	内码 Lagrange	外码 Lagrange	共享代数结构
Baseline	—	否 (BM)	无
方案 A	是 (LLOSD)	否 (LCC-BR 传统实现)	部分
方案 B	是 (LLOSD)	是 (LCC-BR + Lagrange 共享)	全共享

Table 1: R1 引出的三级消融实验

对整体方案的影响：

- 需要多做做一个外码译码器：Lagrange 版本的 RS 软判 (与内码 LLOSD 共享 α^j 表、denominator products、Lagrange 基函数 $T_j(\alpha^i)$)。
- 汇报时对比：Baseline / 方案 A / 方案 B 三条时延曲线，从共享粒度上量化收益。
- 预期效果：方案 B 相对方案 A 能省去 $O(n \cdot k')$ 次 $\mathbb{F}_{2^m}$ 乘法 (Lagrange 基函数表的重建成本)。

1.2 R2 ——10% 时延 KPI 对两个基线都要满足

R2 原文：10% 时延是分别 RS 硬判决和软判决的时延基线。针对时钟周期和 $\mathbb{F}_{2^m}$ ops。

解读：这条明确了 KPI 的具体形式：

硬性 KPI

- $T_{\text{RS+BCH}} \leq 1.10 \times T_{\text{RS-BM}}$ (Case a: 硬判基线)
- $T_{\text{RS+BCH}} \leq 1.10 \times T_{\text{RS-LCC-BR}}$ (Case b: 软判基线)

两个 case 都需要满足，且两种测量口径 (时钟周期 & $\mathbb{F}_{2^m}$ ops) 都要报告。

初步分析 (基于 IEEE TIT 2025 的 Table IV 及 PLCC 论文的实测)：

译码器 @ 5 dB	$\mathbb{F}_{2^m}$ ops	相对 BM 倍数	是否满足 Case a	是否满足 Case b
RS-BM (n=255)	$\sim 3 \times 10^3$	1.0×	—	—
RS-LCC-BR (n=255)	$\sim 3.75 \times 10^5$	125×	—	—
BCH-LLOSD(2) (n=255)	$\sim 1.9 \times 10^4$	6.3×	× (>10%)	✓ (≤10%)
BCH-HSD(1,8) (n=255)	$\sim 9.9 \times 10^3$	3.3×	× (需极致优化)	✓ (≤10%)

Table 2: 两种 KPI 基线下 BCH 内码时延占比初步估算 (n=255)

Case a 的可行性分析

结论 A：Case b (相对 RS 软判基线) 满足 10% 预算相对容易，因为 LCC-BR 本身开销就是 BM 的 $\sim 100\times$ ，BCH 加进去 (相当于 BM 的 $\sim 3-6\times$) 只占 3-6%，留有充足 buffer。

结论 B：Case a (相对 RS 硬判基线) 从 $\mathbb{F}_{2^m}$ ops 层面几乎不可能满足——LLOSD 的运算量本质上是 BM 的 $3-6\times$ ，10% 的预算需要压缩到 BM 的 $0.1\times$ ，差 $30\times$ 以上。可能的应对：

- 用极致的硬件并行度 ($P=64\times$ 并行 $\mathbb{F}_{2^m}$ 乘法器) 才有希望
- 或者转换视角：在时钟周期 (关键路径深度) 下测量而非总 ops 数
- 或者只在高 SNR (6 dB+) 区间达到 (SNR 越高 LLOSD 越快，Table I 显示 6 dB 时可到 BM 的 8%)

1.3 R3 ——n=128 和 n=255 并列做

R3 原文：并列做。可以。

解读：两个码长都要做，工作量翻倍。但真正的挑战在于两个 config 底层参数完全不同：

	n=128 方案	n=255 方案
Galois 域	$\text{GF}(2^7) = \text{GF}(128)$	$\text{GF}(2^8) = \text{GF}(256)$
primitive polynomial	例如 0x83	例如 0x11D
BCH 类型	需 extended BCH (127+1) 或 shortened BCH	primitive BCH ($255 = 2^8 - 1$)
Lagrange 表大小	128×128	256×256
存储	~16 KB	~64 KB

n=128 的技术风险 (需要老师定夺)

论文 LLOSD 的 Theorem 2 只覆盖 **primitive narrow-sense BCH** (码长 $n = 2^m - 1$)。因此 $n = 128$ 需要用：

- **Extended BCH** (加一位全奇偶校验)：需要推广 **Theorem 2** 到扩展码——这是一个小 novelty (需要小心处理 syndrome 计算和过滤条件)

- **Shortened BCH** (删除信息位)：更简单，但码率下降需要重新调整参数

无论哪种做法，都比 $n = 255$ 多做一份专门工作。

1.4 R4 ——外码软判用 LCC-BR

R4 原文：用后者 (LCC-BR)。

解读：LCC-BR 是最合适的外码软判选择，理由：

- 与 LLOSD 同源：论文 §V 的 HSD 算法就是 LLOSD + LCC-BR 集成，两者共享 basis reduction 结构——**Lagrange** 共享的技术路径已经在论文里被打通。
- 复杂度低、硬件友好：Xing-Chen-Bossert 2020 论文中 LCC-BR 的 module basis reduction 比 Kötter-Vardy 的 factorization approach 少约 1/3 的 $\mathbb{F}_2^m$ 运算。
- 与老师的时延 **KPI** 契合：LCC-BR 的低时延特性正是我们要利用的。

工作量：LCC-BR 的核心是 Mulders–Storjohann basis reduction + partial root-finding。Python 复现里当前是简化实现 (用 BM 替代)，Matlab 版需要完整实现，预计工作量 1–1.5 周。

1.5 R5 ——所有译码器自己实现

R5 原文：所有译码器自己实现。

解读：不允许使用 Matlab Communications Toolbox 的 `comm.RSDecoder` / `comm.BCHDecoder` 等任何内置译码器。

好消息：Python 复现代码中所有的算法逻辑都可以 port，工作量集中在语言转换：

预计整体 **port** 工作量：2–3 周 (含单元测试与 Python 版结果对齐)。

1.6 R6 ——三个参数方案都需要

R6 原文：三个方案都需要。

Python 模块	对应 Matlab 模块	转换成本
src/gf.py	GF class + EXP/LOG 查表	低 (纯查表)
src/bch.py	BCH encoder + BM decoder	低 (向量化直接对应)
src/osd.py	OSD baseline (GE + TEP enum)	中 (Matlab GE 有内置)
llosd.py + llosd_jit.py	LLOSD + 内循环	高 (Numba JIT 需用 MEX 或 vectorization 替代)

解读：仿真矩阵扩展如下：

方案	(n, RS, BCH)	码率	域
n=255 方案 A	RS(255, 239) + BCH(255, 239)	0.879	GF(2 ⁸)
n=255 方案 B	RS(255, 235) + BCH(255, 247)	0.891	GF(2 ⁸)
n=128 方案	RS(127, 119) + BCH(127, 120)	0.885	GF(2 ⁷)

仿真矩阵规模：3 参数 × 2 基线 (BM / LCC-BR) × 2 方案 (A/B) × 2 测量口径 (cycles / ops) × 15 SNR points = 约 **360** 个数据点。每个数据点约需 10⁴ 帧收敛 → ~ 4 × 10⁶ 帧总仿真量。

应对：使用 Matlab `parfor` 并行 + 参数化 config file 统一驱动。

2 基于 R1–R6 的三个新技术追问

在动手写代码前，还有三个技术细节希望老师再指点，因为它们会决定后续几周的具体路线：

Q7. Clock cycle 测量的 hardware timing model

纯软件仿真无法直接得到 clock cycles。我建议定义一个参数化的抽象硬件模型：

- 每个 $\mathbb{F}_{2^m}$ 乘法 = 1 cycle
- 允许 P 路并行 (硬件并行度)
- 报告 $P \in \{1, 4, 16, 64\}$ 四种硬件档次下的关键路径 depth

请问这个 model 可以接受吗？还是老师有指定的 timing model？

Q8. Case (a) 的可行性策略

如 §1.2 分析，Case (a) 相对 RS 硬判决 BM 的 10% 时延预算在 $\mathbb{F}_{2^m}$ ops 层面几乎不可能满足 (需将 LLOSD 时延压到 BM 的 0.1×，差 30× 以上)。

请问在这种情况下：

- 是否可以只在时钟周期 (关键路径) 视角下 (即高并行度硬件模型) 满足 10%？
- 是否可以只在中高 SNR (≥ 5.5 dB) 区间满足？
- 还是必须两个基线的 10% 都要在所有 SNR、所有测量口径下无条件满足？

这个方向会决定我要不要花大量精力在极致优化上 (例如 Lagrange 查表化、TEP 循环展开等)。

Q9. $n=128$ 用 Extended BCH 还是 Shortened BCH

$n = 128$ 不是 primitive BCH 长度 ($2^m - 1$)，需要通过 extended 或 shortened 构造。两种方式都会对 LLOSD 的 Theorem 2 需要小幅推广：

- Extended BCH: $n = 128 = 127 + 1$ ，加一位全奇偶校验。Theorem 2 的过滤条件需要加一个额外的 **syndrome** 检查。
 - Shortened BCH: 例如 BCH(127, 119) 用作 (128, ?) 效果不佳；或从 (255, ?) shortened 到 128。
- 请问老师倾向哪种？**Extended BCH** 是更规范但需要小 **novelty**；**shortened** 更简单但码率折损。

3 更新后的工作量估算与执行计划

阶段	时长	交付物
Phase 1	Week 1–2	Matlab 项目框架 + $GF(2^7)/GF(2^8)$ 有限域算术 + BCH 编码 + BM 硬判 + PAM4 modem + AWGN 信道模型。BER-SNR 基线曲线与文献对齐
Phase 2	Week 3–4	LLOSD 从 Python 复现代码 port 到 Matlab (含 SLLOSD 和 ML 提前终止)。BCH 单码仿真结果与原论文 Fig 3/4 对齐
Phase 3	Week 5–6	RS-LCC-BR 完整实现 (Mulders-Storjohann basis reduction + partial root-finding)。外码 RS 单码性能验证
Phase 4	Week 7–8	级联链路搭建 + Lagrange 共享层 (方案 A 和方案 B)。n=255 方案 A 完整仿真 (最小闭环)
Phase 5	Week 9–10	横向扩展: n=255 方案 B + n=128 方案。Extended BCH 的 Theorem 2 推广 (若采用)
Phase 6	Week 11–12	两种测量口径的时延统计、10% KPI 验证、最终报告 + Matlab 代码整理与交付

总工作量估计：**10–12 周**。相比初版的 6–8 周，主要增量来自：

- R6 的三个参数方案 → 仿真组合数 $\times 3$
- R3 的 $n=128$ 需要 **extended/shortened BCH** 处理
- R1 的方案 A / 方案 B 消融对比
- R2 的两个基线 (BM & LCC-BR) $\times$ 两种测量口径 (cycles & ops)

建议的增量执行策略

先做最小闭环 (**n=255 方案 A + LCC-BR baseline + F_{2^m} ops**)，跑通后请老师 review 中间结果和路线正确性。确认后再横向扩展到其他 config。避免同时展开所有维度导致后期发现底层 bug 需要全面 rework。

参考: L. Yang et al., “Efficient OSD of BCH Codes Without GE,” *IEEE Trans. Inf. Theory*, vol. 71, no. 11, pp. 8294–8311, Nov 2025. Python 复现: <https://github.com/a-yeyang/efficient-osd-bch-no-ge>. 文档: 2026-07-23 · 陈诗阳